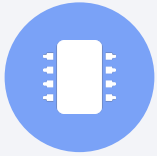


ENME 441

03 – ESP32 and MicroPython

Topics



PART 1

ESP32-C3 SuperMini overview

The microcontroller hardware

- The ESP32 family
- ESP32-C3 SuperMini
- GPIO pinout



PART 2

setup

*Preparing the ESP32 for MicroPython
coding*

- Flashing MicroPython
- Thonny and the REPL
- main.py and the repo



PART 3

micropython

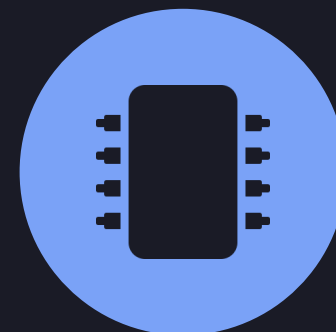
The MicroPython language

- What is missing vs. CPython
- Filesystem and os
- The time module

01

ESP32-C3 SuperMini

Overview of the ESP32-C3 board



Espressif ESP32

A microcontroller is a CPU, memory and I/O on one chip — this one adds a radio.



Processing

Dual-core Xtensa LX6 32-bit CPU @ 160–240 MHz
Up to 8 MB SRAM · up to 16 MB flash



Radios

WiFi 802.11 b/g/n
Bluetooth Classic and BLE



Digital buses

3× UART · 3× SPI · 2× I2C · 2× I2S audio



Analog and I/O

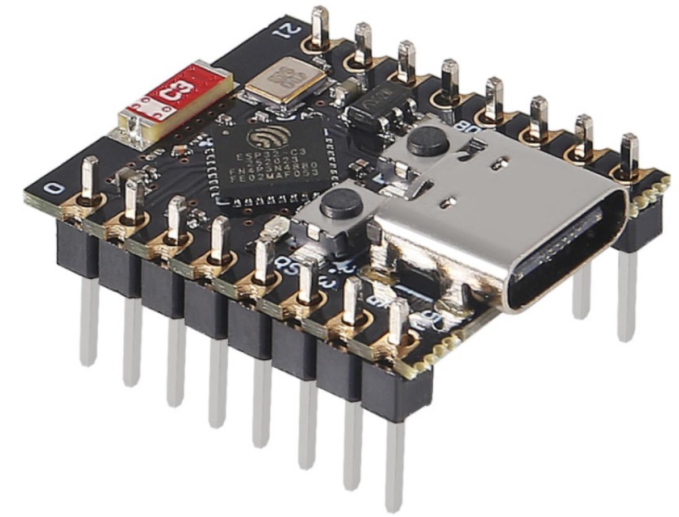
14× GPIO · 12× ADC inputs · 2× DAC outputs
Built-in Hall effect sensor



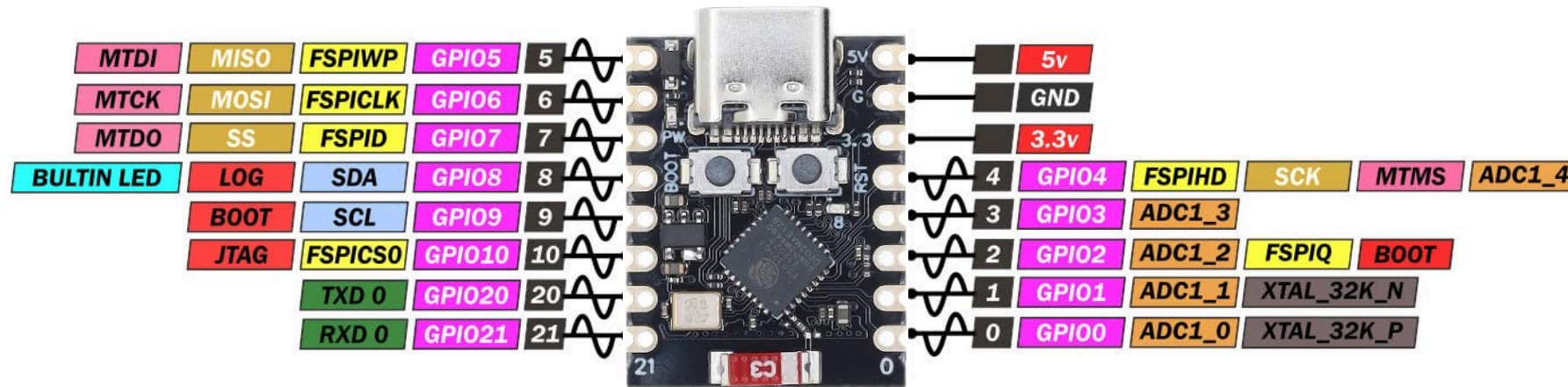
ESP32 module — chip, flash, antenna

ESP32-C3 SuperMini

- **Processor** 32-bit RISC-V single-core CPU, up to 160 MHz
- **Memory** 400 kB SRAM · 384 kB ROM · 4 MB on-board flash
- **Connectivity** 2.4 GHz WiFi 802.11 b/g/n · Bluetooth 5.0 LE
- **USB and power** USB-C for power and serial, 5 V (USB) or 3.3 V pins
- **I/O pins** 13 GPIO including ADC, SPI, I2C, UART and PWM
- **Programming** Arduino IDE, ESP-IDF, PlatformIO, MicroPython



ESP32-C3 Pinout



- JTAG (Joint Test Action Group): standard hardware interface for testing and debugging
- SPI (Serial Peripheral Interface): connect to external peripheral devices (defaults shown, can be changed to any GPIO pins)
- MISO/MOSI: hardware SPI (high-speed serial peripheral interface)
- SDA/SCL: serial data pins for I²C communication (defaults shown, can be changed to any GPIO pins)
- TXD/RXD: hardware UART (Universal Asynchronous Receiver-Transmitter) pins
- XTAL: external crystal oscillator interface (precision timing)
- GPIO: general-purpose input/output pins
- ADC: analog-to-digital conversion
- Built-in LED on GPIO8 (pull-up resistor, so 0 == ON, 1 == OFF)
- GPIO 2/8/9 are boot straps that must be HIGH at reset – never connect these pins to pull-down circuits!

02

Board setup

Preparing the ESP32-C3



MicroPython Firmware Installation

A new ESP32 needs custom firmware to run MicroPython. Firmware can be changed as many times as you like.

1 Download and install Thonny

Thonny is a free Python IDE that speaks to MicroPython boards over USB.

2 While holding BOOT, plug in the USB-C cable

Holding the BOOT button during power-up puts the chip in its download mode, making it ready to accept new firmware.

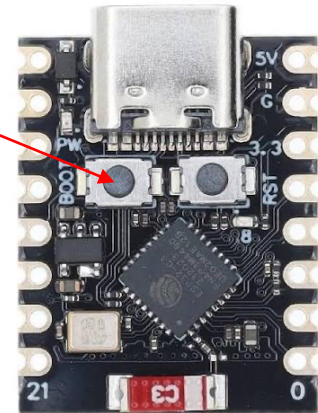
3 Open Tools → Options → Interpreter



Only needed once per board

Reflash only if the firmware is corrupted, or you want a newer MicroPython version. Flashing erases everything on the board, including your own files.

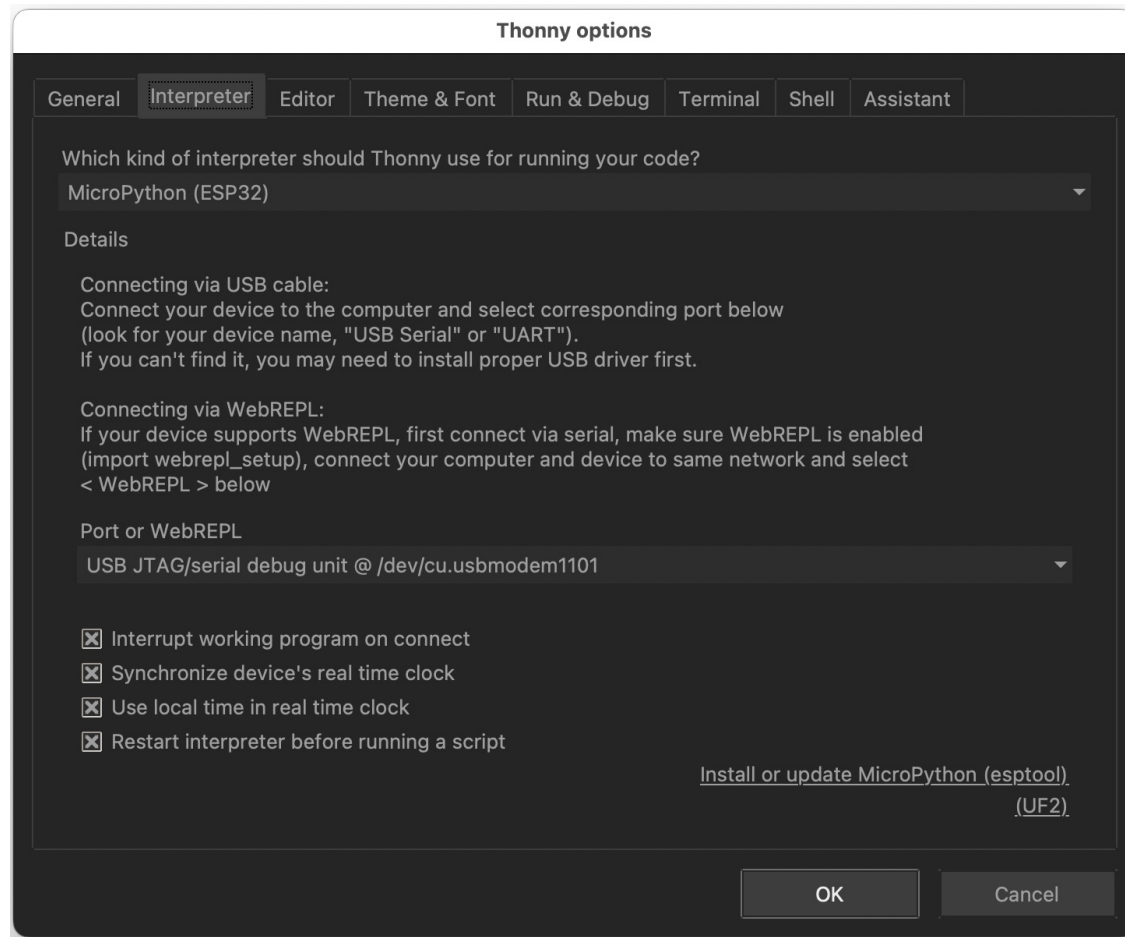
BOOT
button



Step 1: Point Thonny at the Board's USB interface

Tools → Options → Interpreter.

Set the interpreter & port, then start the installer.



- **Interpreter**

→ MicroPython (ESP32)

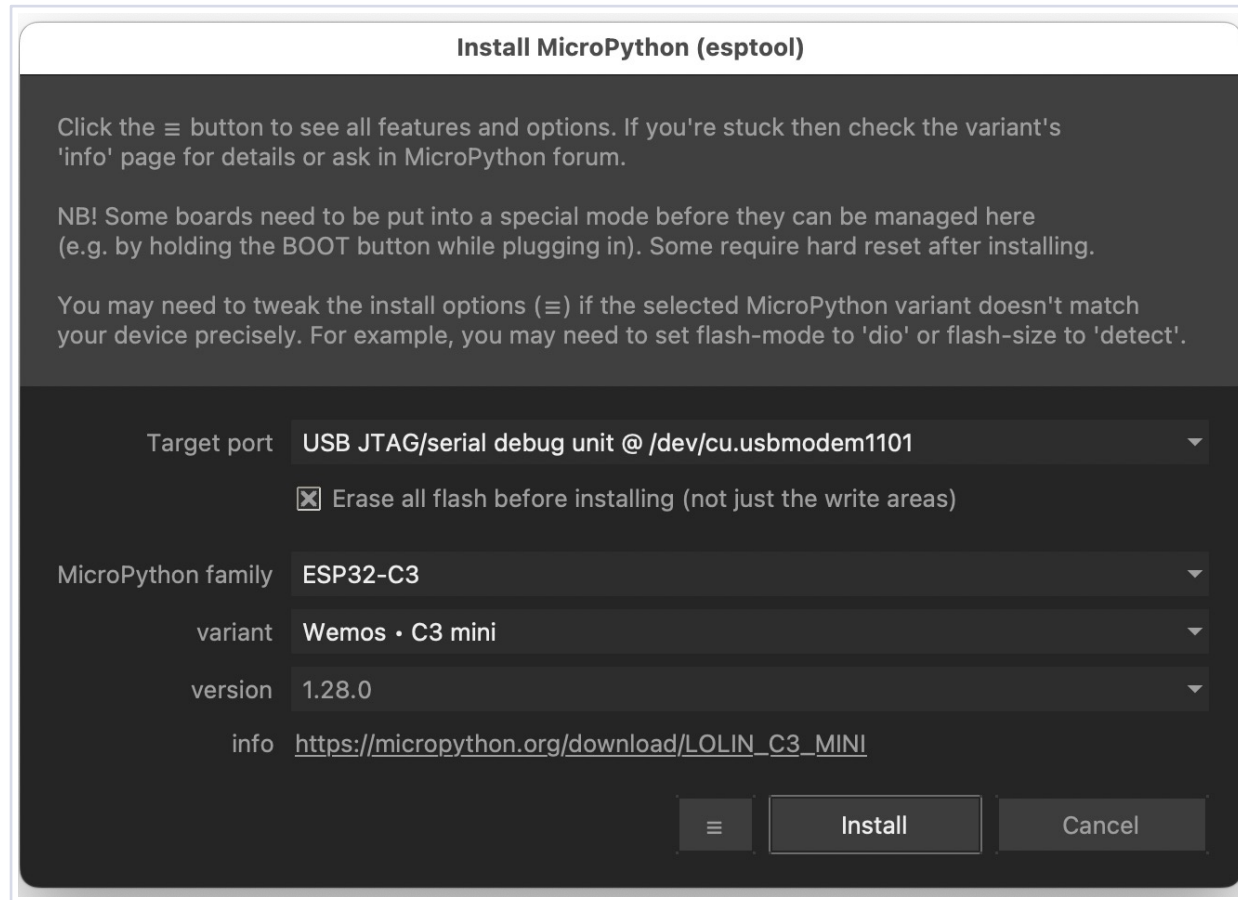
- **Port or WebREPL**

Pick the entry with "USB JTAG" or similar in its name. If nothing appears, the cable is charge-only or the driver is missing.

- **Install or update MicroPython (esptool)**

The link at the bottom right of the dialog. Clicking it opens the installer on the next slide.

Step 2: Install the Firmware



- **Target port**

Use the same port from the previous step

- **MicroPython family**

→ ESP32-C3

- **Variant**

→ Wemos C3 mini

- **Erase all flash before installing**

Leave ticked for a clean install.

- **Click Install**

When it finishes, close both dialogs and return to the main Thonny window.

Step 3: Check That It Worked



Click the red STOP

In the Thonny toolbar. This interrupts whatever the board was doing and hands you the prompt.



Look for the REPL prompt

The shell pane at the bottom should show the MicroPython banner and >>>. If it doesn't, reboot the board.



Type a command

Enter a Python statement at the >>> prompt. If it evaluates, MicroPython is running on the board.

```
Unable to connect to /dev/cu.SLAB_USBtoUART: [Errno 2] could not open port /dev/cu.SLAB_USBtoUART: [Errno 2] No such file or directory: '/dev/cu.SLAB_USBtoUART'

Backend terminated or disconnected. Use 'Stop/Restart' to restart.

MicroPython v1.19.1 on 2022-06-18; ESP32 module with ESP32
Type "help()" for more information.
>>> |
```

MicroPython (ESP32)

After STOP — banner, version and the >>> prompt

```
Unable to connect to /dev/cu.SLAB_USBtoUART: [Errno 2] could not open port /dev/cu.SLAB_USBtoUART: [Errno 2] No such file or directory: '/dev/cu.SLAB_USBtoUART'

Backend terminated or disconnected. Use 'Stop/Restart' to restart.

MicroPython v1.19.1 on 2022-06-18; ESP32 module with ESP32
Type "help()" for more information.
>>> print("interpreter installation successful!")
interpreter installation successful!
>>> |
```

MicroPython (ESP32)

A command entered at the prompt, and its output

03

WiFi setup on umd-iot

Getting the ESP32 onto the umd-iot network



ENME441 MicroPython Code Repository

Reminder: All course code is available from Github.



`github.com/ddevoe-umd/enme441`

1

Code is updated throughout the semester

Corrections and additions land as we go, so pull again rather than assuming the copy you took in week 2 is current.

2

Keep copies of your own code on your laptop or in the cloud

The board is not backed up! What lives only in ESP32 flash is one reflash or stray gamma particle away from being gone forever.

3

Use mip to copy files from the repo to your ESP32

Unlike your laptop, the ESP32 has no OS to run git commands from. Instead, we will use the [mip MicroPython package manager](#) to copy files from GitHub to your microcontroller.

Why Not eduroam?

The campus network your laptop uses is the wrong network for a microcontroller.

1

eduroam is WPA2/WPA3-Enterprise (802.1X)

It needs per-user credentials and a supplicant that can do certificate-based EAP authentication.

2

MicroPython's WLAN driver only speaks WPA2-Personal

`network.WLAN()` takes an SSID and a pre-shared key — there is no place to put a username, certificate, or EAP method.

3

eduroam ties the connection to a person

Credentials expire; a headless device needs a stable, non-expiring connection.

4

umd-iot exists exactly for this

The umd.iod network authenticates by registered MAC address with a per-device password — something the ESP32 can actually do.

Step 1 — Prepare key files on your ESP32

No network connection yet, so can't use mip for file access

For now, the file transfer sequence is Github → Laptop → ESP32

1

Find 4 files on your laptop from the cloned Github repo (see Intro slides)

[path_to_repo]/enme441/micropython/03_ESP32_and_MicroPython/
get_mac_address.py, main.py, wifi.py, config.py

2

Run Thonny with the ESP32 connected

The REPL prompt should appear — if not, press STOP. If still not, check the USB port (lower right corner of Thonny.)

```
MicroPython v1.28.0 on 2026-04-06; LOLIN_C3_MINI with ESP32-C3FH4  
Type "help()" for more information.
```

```
>>>
```

3

Copy all 4 files to the ESP32

View → Files in Thonny to open the file browser panel

Navigate to the file directory on your laptop

Hold Ctrl (Windows) or Cmd (Mac) to select multiple files on your laptop

Right-click any of the selected files and click 'Upload to /'

FILES TO COPY

- `get_mac_address.py`
- `main.py`
- `wifi.py`
- `config.py`



Do not rename the files

The file names must stay unchanged on the ESP32

main.py

```
import time
import config
import wifi

for _ in range(3):
    print('.', end='')
    time.sleep(0.5)
print('ESP32 C3 SuperMini booted up\n')

wifi.connect()
```

- **boot.py and main.py run automatically at ESP32 power-up.**

Code in main.py will execute without having to interact with the microcontroller.

- **Change code in main.py only**

Both files can be modified, but boot.py should be reserved for low-level board setup. Both share one global context, so names set in boot.py are visible in main.py.

- **Keep your code modular**

Import modules into main.py rather than running code within the file itself.



Thonny misses the boot sequence when you first plug the board in. To see main.py's output in the shell, press STOP then ctrl-D for a soft reboot, which keeps the serial connection.

wifi.py

`wifi.connect_wifi()` tries each known network in turn, then gives up

```
def connect():
    wlan = network.WLAN(network.STA_IF)
    wlan.active(False); time.sleep(0.5)
    wlan.active(True); time.sleep(0.5) # radio up

    if wlan.isconnected():
        print('WiFi connected:', wlan.ifconfig())
        return

    for ssid, password in config.WIFI_NETWORKS:
        wlan.connect(ssid, password)
        start = time.time()
        while not wlan.isconnected():
            if time.time() - start > 5:
                wlan.disconnect()
                break # -> try next network
            time.sleep(1)
        else:
            print('WiFi connected:', wlan.ifconfig())
            return
```

- **Power-cycle the radio**

`active(False)` then `active(True)` clears a stale association left over from a previous boot.

- **while ... else**

The `else` clause runs only if the `while` loop finishes without breaking, i.e. a network has connected.

- **5-second budget per network**

`break` drops to the next entry in `WIFI_NETWORKS` instead of blocking forever.

- **Non-blocking by design**

`connect()` returns immediately; `isconnected()` is what you poll.

get_mac_address.py

Find the Media Access Control (MAC) hardware address for your ESP32

```
# get_mac_address.py
#
# Find the MAC address for your ESP32 to register with
# the umd-iot network

import network
import ubinascii

wlan = network.WLAN(network.STA_IF)
wlan.active(True)
mac = ubinascii.hexlify(wlan.config('mac'), ':').decode()
print('MAC address:', mac)
```

Run the code on your ESP32 and record the result:

```
>>> %Run -c $EDITOR_CONTENT
MAC address: 34:85:18:0a:1b:2c
```

- **network.STA_IF**

This is the station (client) interface that joins a network, not the interface that creates a network (AP_IF).

- **active(True) first**

The radio has to be powered up before config('mac') will return anything.

- **hexlify(..., ':')**

Formats the six raw bytes as the colon-separated string the UMD portal expects.

- **MAC address is unique to each board**

Six bytes stored in ROM uniquely identifies every network interface device.

- **1. Run the code on your ESP32**

Open get_mac_address.py in Thonny and press Run (F5). The address prints in the Shell.

- **2. Record the address**

Copy and save your MAC address

Step 2 — Register your ESP32 on umd-iot

You must be on campus WiFi (or using the campus VPN) to reach the portal.

1

Go to <https://it.umd.edu/connect>

Log in with your university credentials to access the IoT device registration portal.

2

Choose "Gaming or Streaming Device [...]" Register your device for umd-iot"

3

Click on "Create Device"

4

Copy and paste your MAC address from Step 1

Name your device for future reference. The name only matters as an identifier for the umd-iot registration system.

5

Copy the generated "umd-iot Wi-Fi Password"

This is the unique password tied to your specific ESP32 for logging in to the umd-iot network

Step 3 — Edit credentials in config.py

`config.py` is the only file you edit during setup; `wifi.py` imports from it.

Use `config.py` to hold all global variables and constants to be shared across all modules.

```
# config.py -- all cross-module constants

UMD_IOT_PASSWORD = 'your_umd_iot_password'

# (ssid, password) pairs -- add more for off-campus
WIFI_NETWORKS = [('umd-iot', UMD_IOT_PASSWORD)]
```

- **Enter your umd-iot password from Step 2**

Modify `UMD_IOT_PASSWORD` based on the password provided during the IoT device registration process.

- **WIFI_NETWORKS is a list of tuples**

You can add one or more networks as a additional (ssid, password) tuples.

- **Never share this file!**

It holds passwords. Keep it off GitHub.

Step 4 — Set up mip

mip is MicroPython's pip package installer.

Use mip to download all future course code directly to the ESP32 from Github.

1 Reboot your ESP32 to connect with umd-iot

Soft-reboot (ctrl-D in Thonny) or hard-reboot (reset button or pull power)

2 `import mip`

You must import the module each time before use.

3 Use `mip.install()` to download I2C scanner code from the repo

`i2c_scanner.py` is small utility to scan the ESP32's I2C bus.

`mip.install()` requires a direct link to the code file, see the full path below.

● Files land in `/lib`

`/lib` is on the standard micropython import path.

Pass `target='/'` as a second argument if you want to install to the root directory instead.

Thonny REPL — once WiFi is up

```
import mip

mip.install('github:ddevoe-umd/enme441/micropython/03_ESP32_and_MicroPython/i2c_scanner.py')

import i2c_scanner
i2c_scanner.scan(8,9)
```

Scan the I2C bus

I2C is a two-wire communication bus that supports multi-leader + multi-follower comms. We will talk more about I2C when covering communication buses. For now, just verify that your downloaded code can run.

```
from machine import I2C
from machine import Pin

i2c = I2C(sda=Pin(8), scl=Pin(9))

print('Scanning I2C bus...')
devices = i2c.scan()
print(f'I2C device count: {len(devices)}')
for d in devices:
    print(f'Address: {hex(d)}')
```

- Pins 8 and 9 are only the defaults — pass different ones to `I2C()` if your wiring needs it.
- Addresses are returned by `scan()` as integers; `hex()` prints them as hexadecimal values.

WiFi: Things to Watch For



"Connecting to WiFi..." hangs forever

Typically a MAC address error. Re-run `get_mac_address.py` and compare it character by character with the `iot.umd` portal entry.



Using the wrong password

The `umd-iot` password is generated by the registration portal and is specific to that one device. This is NOT your standard UMD directory password!



Connects, but no internet

`umd-iot` can take a few seconds to provision a newly registered device. Reset the board; if it persists, confirm that your device's `umd.iot` registration is still active.



Each new board needs its own registration

Registration is per MAC address. A new board will have its own MAC address — register the new ESP32 and update `config.py` on that board.

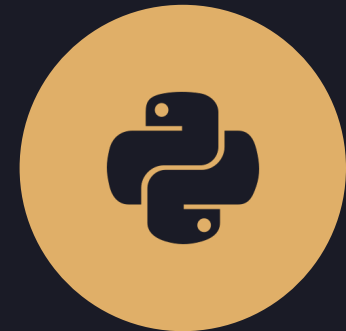
Lab 3 – ESP32 Setup

[See the Lab 3 assignment](#)

03

micropython

Python cut down for microcontrollers



MicroPython — Python for Microcontrollers

An implementation of Python 3.5+ for microcontroller hardware, optimised for small memory and limited hardware.

1

It is a real Python, not a lookalike

The syntax, the data structures and the control flow from the previous lecture all carry over unchanged.

2

Still under active development

Which means thin documentation in places, and occasional bugs. Check the docs before assuming a feature behaves as it does on your laptop.

3

It does not support everything

Read the documentation for the specific operator, function or library you want — the limitations are per-feature, not across the board.

4

esp and esp32 add the hardware

Those modules, along with machine, expose the parts of the chip that have no equivalent in CPython.



`docs.micropython.org`

MicroPython vs. CPython

Where the Python you know stops matching the Python on the board.



No numpy, pandas or matplotlib

The scientific stack is not available. For array maths there is ulab, a numpy-shaped subset built into many firmware images.



Standard modules ship as "micro" versions

utime, ujson, uos, usys, urequests — same names as the originals with a u prefix, and fewer features behind them.



Regular expressions are lightweight

The re module exists, but with no lookbehind and only limited backreferences.



String formatting mostly works

.format() and f-strings behave as you expect — a few edge cases do not.



Memory fragmentation is real

Repeatedly allocating and freeing objects can raise MemoryError on a board that has plenty of free bytes, just not contiguous ones.



No multiprocessing, no subprocess

There is no shell to shell out to, and no environment variables in the Unix sense. File I/O works, but is limited.

MicroPython File System

The board has a real filesystem in the 4 MB of flash

```
import os

os.listdir()    # list the contents of the current directory
os.remove()     # delete a file
os.rename()     # rename a file

os.getcwd()     # get the current working directory
os.chdir()      # change the current directory

os.mkdir()      # create a new directory
os.rmdir()      # delete a directory
```

Reference: docs.micropython.org/en/latest/library/os.html

- **Standard Python file calls work**

`open()`, `read()`, `write()`, `close()` all behave as they do on your laptop.

- **But with `open()` is not supported**

The context-manager form does not exist here. Open the file, use it, then close it yourself.

- **4 MB is not much**

It holds your code comfortably and data files barely. Anything large belongs off the board.

Notes on ESP32 Files



Keep a copy on your laptop (or your own Github account)

Not just on the ESP32. A reflash wipes the board, and boards get reflashed.



Back it up somewhere else too

Box, Google Drive, GitHub — anywhere that is not the one machine you are working on.



Keep the board lean

Only the code that needs to run should be on it. Everything else is flash you cannot spare.

The time Module

`time` is an alias for `utime` — either name works. These are the functions worth knowing.

FUNCTION	WHAT IT DOES
<code>time.sleep(t)</code>	Sleep <code>t</code> seconds
<code>time.sleep_ms(t)</code>	Sleep <code>t</code> milliseconds
<code>time.sleep_us(t)</code>	Sleep <code>t</code> microseconds
<code>time.ticks_ms()</code>	Time-stamp value with millisecond resolution
<code>time.ticks_us()</code>	Time-stamp value with microsecond resolution
<code>time.ticks_cpu()</code>	Time-stamp value at maximum system resolution
<code>time.time()</code>	Seconds since the last boot
<code>time.time_ns()</code>	Nanoseconds since the last boot
<code>time.ticks_diff(t1, t2)</code>	Difference between two tick values
<code>time.ticks_add(t, d)</code>	Tick value of <code>t</code> plus <code>d</code> ticks

`ticks_diff()` and `ticks_add()` exist to avoid the wrap-around errors you get from calculating deadlines with raw system time — next slide.

Using `ticks_diff()` for Timing

A loop that must run for T seconds. The obvious way has a bug that shows up rarely — and therefore at the worst time.

Using `time.time()`

Take a time stamp, then compare the current time against it on every pass:

```
timestamp = time.time()

while time.time() < timestamp + T:
    # do something
```

Each time value is an integer, and at some point it wraps around to zero. When it does, this comparison is wrong for one cycle — the loop ends early, or runs far too long.

Using `time.ticks_diff()`

`ticks_diff()` tracks the wrap-around for you:

```
tickstamp = time.ticks_ms()

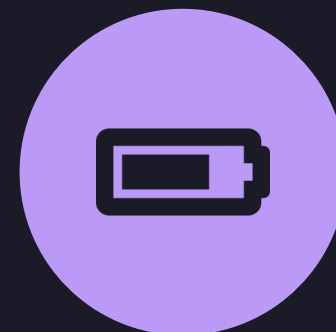
while time.ticks_diff(
    time.ticks_ms(), tickstamp) < 1000*T:
    # do something
```

Note the units: `ticks_ms()` counts milliseconds, so a duration T in seconds becomes $1000*T$. Never subtract two tick values yourself — use `ticks_diff()` to handle wraparound errors.

04

ESP32 power management

Optimizing battery performance using sleep modes



Sleep Modes and Power Consumption

Deactivate hardware blocks to save battery power



Active mode

160 ~ 260 mA

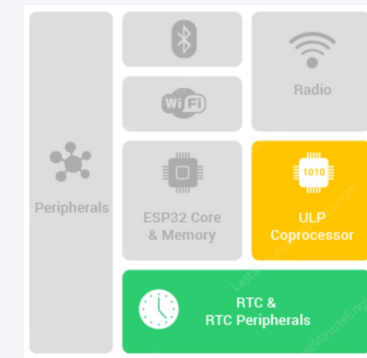
- Normal operation
- All components active at all times



Light sleep

0.8 mA

- WiFi, Bluetooth and radio wake at set intervals
- RAM and CPU are clock-gated
- Timed or GPIO-based wake-up



Deep sleep

0.01 mA

- CPU powered down
- Main memory disabled — only RTC memory is kept
- Timed or GPIO-based wake-up
- The board reboots on wake-up

What That Buys You

The same 2000 mAh LiPo battery, in each of the three modes.

Active mode

2000 mAh / 260 mA

7.7 hours



Light sleep

2000 mAh / 0.8 mA

2500 hours · 104 days



Deep sleep

2000 mAh / 0.01 mA

20 000 hours · 833 days · 3.1 years



Bars are log-scaled — on a linear axis the first two would be invisible. Deep sleep is not a small saving over active mode; it is a factor of 26 000.

Sleep Mode Setup

Two calls to go to sleep, and a set of calls to decide what wakes you up.

INITIATE SLEEP

```
machine.lightsleep(MILLISECONDS)

machine.deepsleep(MILLISECONDS)
```

The duration is optional. Without it the board sleeps until a wake-up source fires — so make sure you have set one.

SET WAKE-UP SOURCES

```
esp32.wake_on_touch(True | False)

esp32.wake_on_ext0(
    pin,
    esp32.WAKEUP_ALL_LOW | esp32.WAKEUP_ANY_HIGH )

esp32.wake_on_ext1(
    pins_list,
    esp32.WAKEUP_ALL_LOW | esp32.WAKEUP_ANY_HIGH )
```

1

ext0 takes one pin, ext1 takes a list

Use ext1 when any of several buttons or sensors should bring the board back.

2

Deep sleep reboots on wake

Execution restarts from `boot.py`, not from where you called `deepsleep()`. Anything you need to survive goes in RTC memory.

Next Steps



Communication Buses

Hardware and software options for ESP32 communications



GPIO

Working with Pin objects for digital input and output.